Documentar Server Express

Paquetes para documentar

`jsdoc`

Herramienta para generar documentación a partir de comentarios en el código.
Estos comentarios deben de tener etiquetas que comienzan con `@`

`swagger-jsdoc`

La herramienta `swagger-jsdoc` creará la documentación de nuestra Api
para ello buscará bloques de comentarios con las etiquetas `@swagger` o `@openapi`
Para crear los bloques de comentarios podemos usar el auto completado de los editores tecleando `/**+Tab`
Dentro de los comentarios podemos indentar usando dos o cuatro espacios **(no usar tabulaciones)**
Las definiciones se escriben en YAML dentro de los comentarios JsDoc

`swagger-ui`

Crea una página web con la documentación de nuestra API a partir de `swagger-jsdoc`

Documentar Server Express

El paquete `swagger-jsdoc` lee los bloques de comentarios para producir un objeto (Json) con la especificación OpenAPI de nuestra API.

La especificación OpenAPI la podemos tener en `Json` o `Yaml`, los dos formatos son válidos ya que un objeto `Json` puede representarse en `Yaml` y viceversa.



```json
{
  "openapi": "3.0.3",
  "info": {
    "title": "Swagger Petstore - OpenAPI 3.0",
    "description": "This is a sample Pet Store Server based on the OpenAPI 3.0
    "termsOfService": "http://swagger.io/terms/",
    "contact": {
      "email": "apiteam@swagger.io"
    },
    "license": {
      "name": "Apache 2.0",
      "url": "http://www.apache.org/licenses/LICENSE-2.0.html"
    },
    "version": "1.0.11"
  },
  "externalDocs": {
    "description": "Find out more about Swagger",
    "url": "http://swagger.io"
  },
  "servers": [
    {
      "url": "https://petstore3.swagger.io/api/v3"
    }
  ],
  "tags": [
    {
      "name": "pet",
      "description": "Everything about your Pets",
      "externalDocs": {
        "description": "Find out more",
        "url": "http://swagger.io"
      }
    },
```

```yaml
openapi: 3.0.3
info:
  title: Swagger Petstore - OpenAPI 3.0
  description: |-
    This is a sample Pet Store Server based on the OpenAPI 3.0 specification.  You can find
    Swagger at [https://swagger.io](https://swagger.io). In the third iteration of the pet
    You can now help us improve the API whether it's by making changes to the definition it
    That way, with time, we can improve the API in general, and expose some of the new featu

    _If you're looking for the Swagger 2.0/OAS 2.0 version of Petstore, then click [here](ht

    Some useful links:
    - [The Pet Store repository](https://github.com/swagger-api/swagger-petstore)
    - [The source API definition for the Pet Store](https://github.com/swagger-api/swagger-
  termsOfService: http://swagger.io/terms/
  contact:
    email: apiteam@swagger.io
  license:
    name: Apache 2.0
    url: http://www.apache.org/licenses/LICENSE-2.0.html
  version: 1.0.11
externalDocs:
  description: Find out more about Swagger
  url: http://swagger.io
servers:
  - url: https://petstore3.swagger.io/api/v3
tags:
  - name: pet
    description: Everything about your Pets
    externalDocs:
      description: Find out more
      url: http://swagger.io
  - name: store
```

Estructura de la especificación OpenAPI

La especificación OpenApi define una estructura de propiedades (Objetos) necesarias para crear una API.
Alguna de estas propiedades son:

- Objeto `Info`: Versión de la API, su nombre, etc,
- Objeto `Contact`: Información de contacto del proveedor/creador de la API
- Objeto `License` Licencia de operación de la API.
- Objeto `Server` Host, URL, Puerto.
- Objeto `Components` Componentes reutilizables en la API.
- Objeto `Paths` Endpoints de la API
- Objeto `Path Item` Operaciones HTTP en un Endpoint (GET, POST, PUT, etc)
- Objeto `Operation` Parámetros y respuestas para una Operación

Usos de la Especificación Open API

El objetivo de OpenApi es proporcionar un estándar para describir APIs REST.
Y a partir de esta descripción/especificación poder obtener:

- La documentación de la API
- Código para consumir la API
- Generación de casos de prueba

Ventajas de la especificación OpenApi

- Principalmente proporciona un estándar para desarrollar/Documentar/Especificar APIs REST.
- Humanos y Máquinas pueden leer la definición y entender como funciona la API sin necesidad de ver su código.
- La definición de la API es independiente del lenguaje de implementación
- Creación de la documentación de manera automática en diferentes formatos HTML, PDF, etc.

Enlaces con la especificación OpenApi

Swagger

Swagger https://swagger.io/specification/

SwaggerEditor https://editor.swagger.io/

OpenApi Initiative

Iniciativa OpenApi https://spec.openapis.org/oas/latest.html

Iniciativa OpenApi Rep.GitHub https://github.com/OAI/OpenAPI-Specification/blob/main/versions/3.1.0.md

Mapa Visual del Objeto Open Api

Mapa visual https://openapi-map.apihandyman.io/

Definición Inicial de de la API

YAML

```yaml
openapi: Versión de la especificación OpenApi que se va usar
info:
  title: Título de la API
  description: Descripción opcional de la API
  version: Número de versio nde la API
servers:
  - url: http://servidorPrincipal/
    description: Descripción del servidor
  - url: http://ServidorSecundario.com
    description: Descipción del servidor
```

JSON

```json
{
  "openapi": "Versión de la especificación OpenApi que se va usar",
  "info": { "title": "Título de la API",
            "description": "Descripción opcional de la API",
            "version": "Número de versio nde la API"},
  "servers": [{"url": "http://servidorPrincipal/",
               "description": "Descripción del servidor"},
              {"url": "http://ServidorSecundario.com",
               "description": "Descipción del servidor"}]
}
```

Propiedad OpenApi	Descripción
`openapi`	Es una cadena con la versión de la especificación OpenApi
`info`	Es un objeto con información acerca de la API como título, descripción, versión
`servers`	Es un arreglo que contiene la(s) URL(s) de la API

Documentación básica de una ruta

Una vez configurado el objeto de definición de la API, solo queda documentar las rutas.
Las rutas o paths son los endpoints o recursos que expone nuestra Api. ó

```
/**
* @swagger
* /usuario:
*   get:
*     summary: Obtener Usuarios
*     description: Devuelve arreglo de usuarios
*     produces:
#       - application/json
*     responses:
*       200:
*         description: Regresa un arreglo
*         type: json
*/
app.get("/usuario", (req, res, next) => {

}
```

Documentación de Parámetros de Ruta

Ejemplo de documentación de una ruta con parámetros en el path `GET /usuario/1`

```
/**
* @swagger
* /usuario/{id}:
*   get:
*     summary: Obtener Usuarios
*     description: Devuelve arreglo de usuarios
*     produces:
*       - application/json
*     parameters:
*       - in: path
*         name: id
*         description: ID del usuario
*     responses:
*       200:
*         description: Regresa un arreglo
%         type: json
*/
app.get("/usuario/:id", (req, res, next) => {

}
```

Documentación de parámetros de cadena de consulta

Ejemplo de documentación de una ruta que acepta parámetros query string. `GET /usuario?id=1`

```
/**
* @swagger
* /usuario:
*   get:
*     summary: Obtener un Usuario
*     description: Devuelve un usuarios
*     produces:
*       - application/json
*     parameters:
*       - in: query
*         name: id
*         description: ID del usuario
*     responses:
*       200:
*         description: Regresa un usuario
*         type: json
*/
app.get("/usuario", (req, res, next) => {
    console.log(req.query.id);      // Parámetro recibido en la url (query string)
}
```

Definición de componentes re-utilizables

Para no tener que reescribir en los bloques de documentación la estructura de los diferentes objetos (recursos) que recibe o devuelve nuestra API, deberemos agregar una sección al final con la estructura de los mismos:

```
/**
* @swagger
* components:
*   schemas:
*     usuario:
*       type: object
*       properties:
*         id_usuario:
*           type: string
*         nombre:
*           type: string
*         ap_paterno:
*           type: string
*/
```

Definición de componentes re-utilizables Con Ejemplos

Para no tener que reescribir en los bloques de documentación la estructura de los diferentes objetos (recursos) que recibe o devuelve nuestra API, deberemos agregar una sección al final con la estructura de los mismos:

```
/**
* @swagger
* components:
*   schemas:
*     usuario:
*       type: object
*       properties:
*         id_usuario:
*           type: number
*           example: 10
*         nombre:
*           type: string
*           example: Gerardo
*         ap_paterno:
*           type: string
*           example: Pineda
*/
```

Referenciando componentes reutilizables en una ruta (En el body en petición POST)

```
/**
* @swagger
* /usuario:
*   post:
*     summary: Alta de Usuario
*     description: Devuelve el resultado de la alta
*     produces:
*       - application/json
*     parameters:
*       - in: body
*         name: Usuario
*         description: Usuario a dar de alta
*         schema:
*           $ref: '#/components/schemas/usuario'
*     responses:
*       200:
*         description: Regresa un objeto con datos de estado
*         type: json
*/
app.post("/usuario", (req, res, next) => { }
```

Referenciando componentes re-utilizables en una ruta (Respuesta en petición GET)

```
/**
* @swagger
* /usuario/{id}:
*   get:
*     summary: Obtener Usuario
*     description: Devuelve un usuarios
*     produces:
#       - application/json
*     responses:
*       200:
*         description: Regresa un usuario
*         content:
*           application/json:
*             schema:
*               $ref: '#/components/schemas/usuario'
*/
```

Agrupación de los recursos en etiquetas

Creación de una sección de tags y a cada ruta asignarle su tag correspondiente:

```
/**
* @swagger
* tags:
* - name: usuario
*   description: Catalogo de usuarios
*/
```

```
/**
* @swagger
* /usuario:
*   get:
*     tags:
*       - usuario
*     summary: Obtener Usuarios
*     description: Devuelve arreglo de usuarios
*     produces:
#       - application/json
*     responses:
*       200:
*         description: Regresa un arreglo
*         content:
*           application/json:
*/
```

Agregar Archivo `Readme.md` en la descripción

1 Crear archivo `README.MD` con una descripción de la Api

```
Descripción de la Api
Descripción de la Api
```

2 Cargar el archivo `README.MD` en una variable y pasarla al objeto de definición:

```
const datosReadme = fs.readFileSync(path.join(__dirname,'readme.md'),{ encoding: 'utf8', flag: 'r' });

const swaggerOptions = {
  definition: {
    openapi: "3.0.0",
    info: {
      title: "API Empleados",
      version: "1.0.0",
      description: datosReadme
    },
    servers: [{ url: `http://localhost:${port}` }],
  },
  apis: [`${path.join(__dirname, "index.js")}`],
};
```

Ejemplos de código en las rutas

Podemos agregar ejemplos de peticiones de código en las documentación de las rutas incluyéndolos como markdown en la propiedad `description`:

```
/**
 * @swagger
 * /usuario:
 *   get:
 *     tags:
 *       - Usuario
 *     description: |
 *       ### Consulta de usuarios.
 *       #### **Curl**
 *       ```bash
 *       curl -X GET http://localhost:3000/usuario
 *       ```
 *       #### **Javascript**
 *       ```js
 *        fetch('/empleado')
 *           .then(response => response.json())
 *           .then(data => console.log(data));
 *       ```
 *     responses:
 *       200:
 *         description: Usuario encontrado..
 *       404:
 *         description: Usuario(s) no encontrados.
 */
app.get('/usuario', (req, res, next) => {
```